

Objectif de travail :

En nous inspirant de nos travaux de la R1.01.P2 sur l'efficacité des méthodes, l'objectif de cette S1.02 est de comparer différents algorithmes pour résoudre un même problème. Nous devons donc comparer chaque solution sur le plan de l'efficacité et le temps d'exécution.

Nous avons en point de départ le jeu du Grundy qui nous a été donnée de manière incomplète. L'on a du coder le jeu en entier sauf les coups de l'ordinateur et nous avons dû créer le test d'efficacité. Donc nous devons comparer les efficacités et les temps d'exécution de chaque version d'une méthode récursive, qui, dans la théorie, garantit à l'ordinateur de gagner après une décomposition de chaque possibilité en choisissant les coups gagnants.

Voici les différentes versions que nous avons à programmer :

- Version 0, la version brute : Créer la boucle de jeu avec l'ordi intelligent, mesurer l'efficacité et le temps d'exécution de estGagnante
- Version 1 : Les positions perdantes sont gardées en mémoire dans un ArrayList et donc ne sont pas recomptées à chaque fois
- Version 2 : Les positions gagnantes sont aussi gardées en mémoire dans un ArrayList et donc ne sont pas recomptées à chaque fois
- Version 3 : Conserver les positions et supprimer les tas perdants. La nature d'une situation ne change pas si on lui ajoute ou si on lui enlève une situation (tas) perdante. Cette théorie permet de ne pas recalculer des positions déjà connues.
- Version 4 : Il faut supprimer les couples de positions gagnantes qui ensemble sont perdantes

Pour toutes les versions suivantes, les tests d'efficacité et les temps d'exécution se baseront sur un nombre d'allumettes allant de 3 à 23.

Efficacité et temps d'exécution de la version 0 :

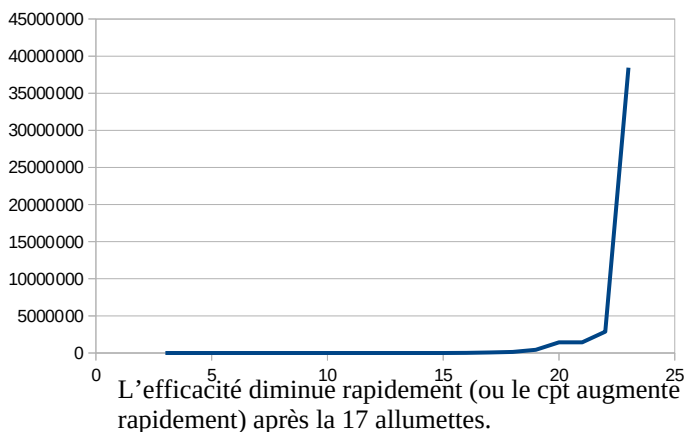
Nous voyons dans ce tableau que les efficacités et les temps d'exécution sont très élevés, la programme vérifie pour chaque tas si les positions sont gagnantes ou perdantes, et cela de façon récursive. Il ne garde rien en mémoire et si il se trouve en face d'une position qu'il a déjà analysé, il va la refaire. Sur un très grand nombre d'allumettes, le programme mettrait énormément de temps avant de trouver la solution.

n	cpt	tps (ms)
3	1	0.011543
4	2	0.011672
5	3	0.01177
6	7	0.020245
7	18	0.047334
8	19	0.058237
9	39	0.106128
10	112	0.299535
11	113	0.344283
12	227	0.578009
13	1267	1.680864
14	2559	1.864703
15	6527	2.290434
16	20141	6.142503
17	62801	15.622409
18	128101	24.540667
19	422885	39.676416
20	1433262	109.238135
21	1433263	109.88321
22	2866527	222.456508
23	38440186	3018.372946

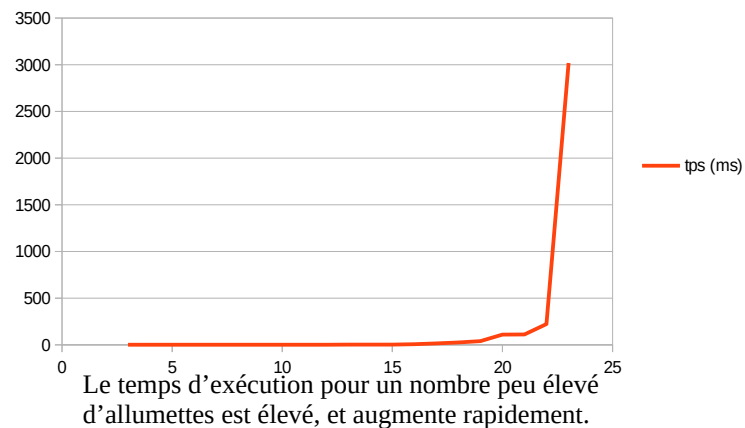
Après plusieurs tests, le temps d'exécution ralentit franchement à partir de l'allumette 20.

La version 0 n'est pas efficace car au bout de 20 allumettes, l'ordi met beaucoup de temps à répondre.

Version 0 : cpt en fonction de n



Version 0 : tps(ms) en fonction de n



On remarque que, parfois, le temps d'exécution ou l'efficacité sont presque identiques d'une allumettes à l'autre, alors que normalement ils doivent être plus élevés. Cela est dû au fait que, parfois mais rarement, rajouter une allumette ne change pas les décompositions et les sous-décompositions effectuées, et les donc, d'une allumette à l'autre, ils ne changent pas.

Des anomalies dans les graphiques sont alors à noter, mais c'est tout à fait normal, et **cela vaut pour toutes les autres version.**

Efficacité et temps d'exécution de la version 1 :

Nous voyons dans ce tableau que les efficacités et les temps d'exécution sont beaucoup moins élevés que la version 0.

Le fait que le programme garde en mémoire les positions perdantes soulage grandement l'efficacité car dès que le programme se trouve face à une position perdante qu'il a déjà analysé, il ne va pas l'analyser, et donc il ne fait pas la récursivité.

Sur un très grand nombre d'allumettes, le programme met beaucoup moins de temps que la version 0, car énormément de positions perdantes sont gardées en mémoire.

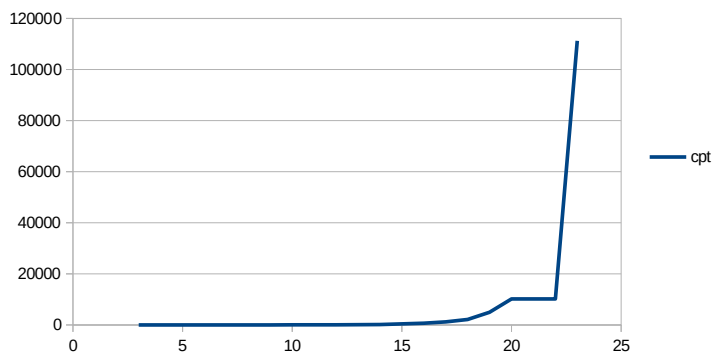
Pour 23 allumettes, $38440186 / 111212 = 345.6$, donc la version 1 est 345.6 fois plus efficace que la version 0.

n	cpt	tps (ms)
3	1	0.228655
4	2	0.020763
5	3	0.015429
6	5	0.024716
7	14	0.085181
8	15	0.056796
9	17	0.05416
10	40	0.109766
11	41	0.119308
12	43	0.139297
13	143	0.509922
14	187	0.524388
15	399	0.789727
16	673	1.021368
17	1221	1.361144
18	2139	1.260266
19	4997	3.814537
20	10176	6.30563
21	10177	7.91056
22	10179	6.446035
23	111212	44.417928

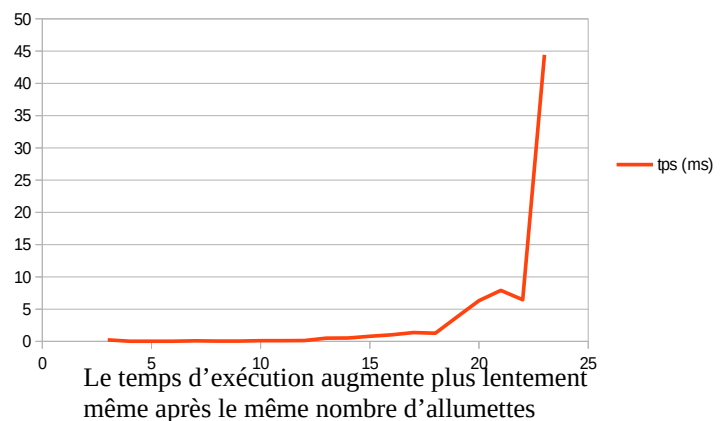
La version 1 est plus efficace car au bout de 22 allumettes, l'ordi est toujours performant.

Après plusieurs tests, le temps d'exécution ralentit franchement à partir de l'allumette 26.

Version 1 : cpt en fonction de n



Version 1 : tps (ms) en fonction de n



Efficacité et temps d'exécution de la version 2 :

Nous voyons dans ce tableau que les efficacités et les temps d'exécution sont moins élevés que la version 1.

Le fait que le programme garde en mémoire les positions gagnantes soulage l'efficacité, mais moins qu'entre la version 0 et 1 car il y a moins de positions gagnantes que de perdantes.

Toutefois, sur un très grand nombre d'allumettes, le programme met beaucoup moins de temps que la version 1, car beaucoup de positions gagnantes sont gardées en mémoire.

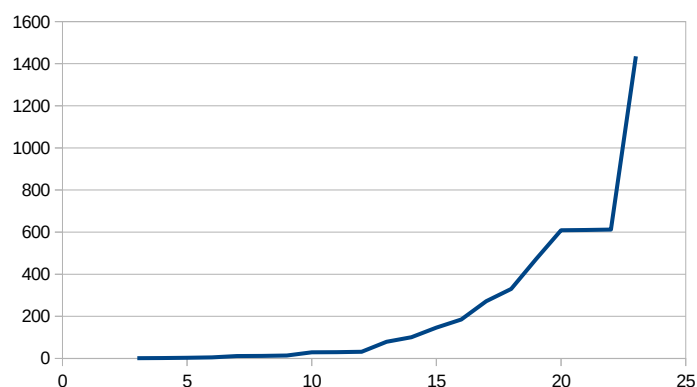
Pour 23 allumettes, $111212 / 1435 = 77$, donc la version 2 est 77 fois plus efficace que la version 1.

n	cpt	tps (ms)
3	1	0.16769
4	2	0.040966
5	3	0.033854
6	5	0.034066
7	11	0.096523
8	12	0.114079
9	14	0.094593
10	29	0.269665
11	30	0.436945
12	32	0.213322
13	79	0.788896
14	101	0.742003
15	146	1.061262
16	185	1.040793
17	272	1.230268
18	330	1.007639
19	472	1.389458
20	609	1.283178
21	610	1.182502
22	612	0.915774
23	1435	3.164323

La version 2 est plus efficace car à 23 allumettes, l'ordi est 77 fois plus performant.

Après plusieurs tests, le temps d'exécution ralentit franchement à partir de l'allumette 36.

Version 2 : cpt en fonction de n



Version 2 : tps (ms) en fonction de n



Le cpt augmente plus lentement que la version 1. Le temps d'exécution est un peu plus chaotique sur ce graphique car on est sur des petites valeurs de temps.

Efficacité et temps d'exécution de la version 3 :

Nous voyons dans ce tableau que les efficacités et les temps d'exécution sont encore moins élevés que la version 2.

Le fait que le programme supprime les tas perdants soulage l'efficacité. A partir d'ici, l'efficacité baisse de moins en moins car le plus gros se fait lorsque l'on garde en mémoire les positions gagnantes et perdantes. Ici on supprime les tas perdants donc cela permet de ne pas recalculer les positions déjà connues

Toutefois, sur un très grand nombre d'allumettes, le programme met un peu moins de temps que la version 2.

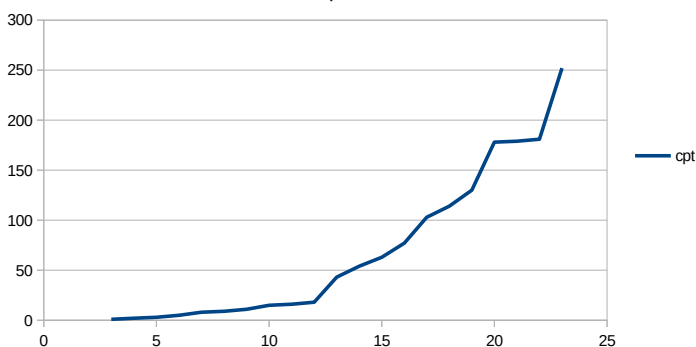
Pour 23 allumettes, $1435 / 252 = 5.69$, donc la version 3 est 5.69 fois plus efficace que la version 2.

cpt	n	tps (nano)
0	1	0
0	2	0
2	3	42761
3	4	58601
4	5	416746
6	6	112613
9	7	176092
10	8	139824
12	9	173928
16	10	177505
17	11	162848
19	12	251755
55	13	955062
85	14	1496003
102	15	1126776
143	16	1329689
184	17	2029860
207	18	1227907
295	19	1744221
346	20	1675672
347	21	1793504
349	22	1484561

Après plusieurs tests, le temps d'exécution ralentit franchement à partir de l'allumette 40.

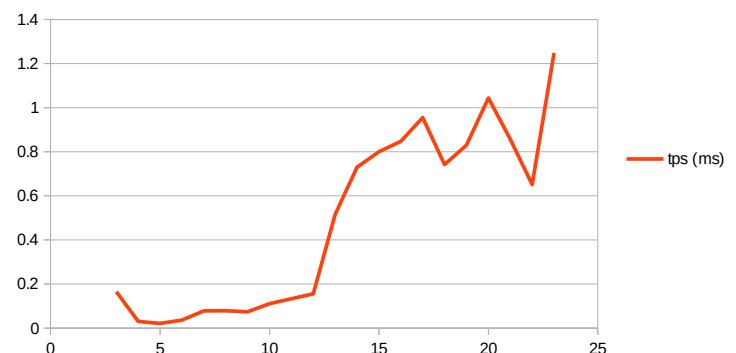
La version 3 est plus efficace car à 23 allumettes, l'ordi est 5.69 fois plus performant.

Version 3 : cpt en fonction de n



Le cpt augmente très lentement, il faut attendre un grand nombre d'allumettes pour qu'il augmente beaucoup

Version 3 : tps (ms) en fonction de n



Le temps d'exécution est un peu plus chaotique sur ce graphique car on est sur des petites valeurs de temps.

Efficacité et temps d'exécution de la version 4 :

Nous voyons dans ce tableau que les efficacités et les temps d'exécution sont un peu moins élevés que la version 3.

Le fait que le programme simplifie les couples gagnants soulage l'efficacité. Ici, il est difficile de faire plus efficace. On supprime les couples de positions gagnantes qui ensemble sont perdantes, donc cela réduit encore plus les tours de boucles à faire.

Sur un très grand nombre d'allumettes, le programme met un peu moins de temps que la version 3.

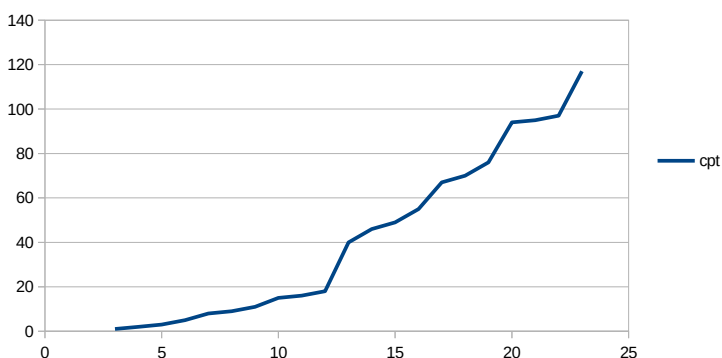
Pour 22 allumettes, $252 / 117 = 2.15$, donc la version 4 est 2.15 fois plus efficace que la version 3.

n	cpt	tps (ms)
3	1	0
4	2	0
5	3	51346
6	5	93775
7	8	463165
8	9	194734
9	11	262120
10	15	302495
11	16	247052
12	18	271167
13	40	284672
14	46	356235
15	49	1027119
16	55	1746013
17	67	1332881
18	70	1166320
19	76	1615188
20	94	1364821
21	95	1386341
22	97	1745302
23	117	1563281

La version 4 est la plus efficace car à 23 allumettes, l'ordi est 328 548 fois plus performant que la version 0.

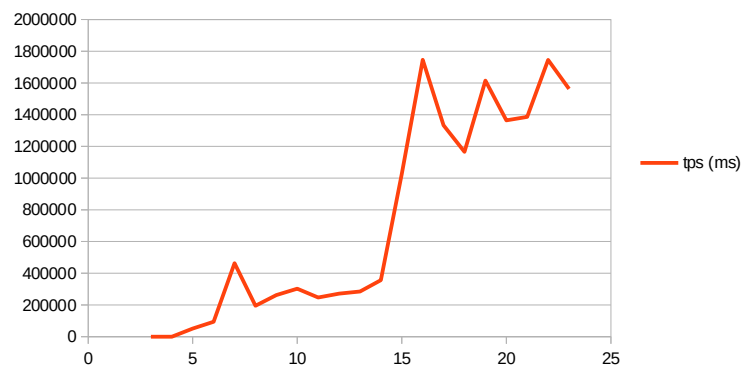
Après plusieurs tests, le temps d'exécution ralentit franchement à partir de l'allumette 54 car le tableau des types est compris que jusqu'à 50, et on ne peut plus simplifier les couples gagnants.

Version 4 : cpt en fonction de n



Le cpt augmente très lentement, il faut attendre un grand nombre d'allumettes pour qu'il augmente beaucoup

Version 4 : tps (ms) en fonction de n



Le temps d'exécution est un peu plus chaotique sur ce graphique car on est sur des petites valeurs de temps.

Conclusion sur l'efficacité et le temps d'exécution des 4 versions :

Au final, la version 4 est (38 440 186 / 117) 328 548 fois plus efficace que la version 0 grâce à la mémorisation des positions perdantes et gagnantes, et grâce à la simplification des positions dans le tableau des positions gagnantes.

C'est important d'être plus efficace car avoir des programmes plus efficaces est améliore les performances, réduit les coûts d'exploitation et offre une meilleure expérience utilisateur.